



SSE206 计算机网络

3-5章知识点梳理

南雨宏

nanyh@mail.sysu.edu.cn

2026-06-17

- 多路复用与分解 (UDP/TCP)
- UDP vs TCP 与应用层控制权
- rdt: 序号、定时器
- GBN vs SR
- 停等协议利用率、流水线模式
- TCP 确认号、累积确认
- TCP 握手/挥手/2MSL
- 流量控制 vs 拥塞控制
- TCP 拥塞控制三阶段
- 数据平面 vs 控制平面
- IP 地址- 二进制地址转换
- NAT 网络地址转换
- 匹配+转发、最长前缀匹配
- IP 分片与 IPv6
- LS vs DV 路由算法
- OSPF 协议
- BGP 协议
- AS 与 iBGP / eBGP
- ICMP 协议

多路复用与多路分解 (UDP / TCP)



■ UDP 多路分解

- 使用二元组 (目的 IP, 目的端口) 定位套接字
- A、B 向 C 同一端口发数据 → 同一套接字; 进程靠源IP/端口区分来源

■ TCP 多路分解

- 使用四元组 (源 IP, 源端口, 目的 IP, 目的端口) 唯一标识连接
- 多客户端连同一 80 端口 → 四元组不同 → 各自独立套接字

UDP vs TCP: 无连接特性



■ UDP 的优点

- 无握手时延、首部仅 8 字节、无拥塞控制、无连接状态

■ 应用层控制权

- 何时发、发多少、是否重传交给应用，便于实时应用自定义可靠性策略

■ DNS 为何默认用 UDP

- 报文短小、一次往返即可、避免握手开销；报文过大或区域传送回退 TCP

■ 取舍

- UDP 灵活但不可靠；在应用层自行实现可靠会引入与 TCP 类似的时延

可靠数据传输 rdt: 序号与定时器的作用



■ 序号 (Sequence Number)

- 解决“重复”：区分新分组与重传的旧分组，识别 ACK 丢失导致的冗余重传

■ 定时器

- 解决“丢失”：分组或 ACK 丢失时触发超时重传

■ 假设RTT 固定且已知，rdt3.0 还需要定时器吗？

- 需要——分组本身可能丢失，根本没有 ACK 会回来
- 定时器是触发重传的唯一机制；去掉它，丢包场景会永久阻塞

流水线协议：GBN vs SR



■ GBN (回退 N 步)

- 累积确认；丢弃乱序分组；超时重传窗口内全部未确认分组；接收窗口 = 1

■ SR (选择重传)

- 逐个确认；缓存乱序分组；只重传真正丢失的分组；收发窗口 > 1
- SR 可能收到落在发送窗口外的 ACK, GBN 不会；序号数 $\geq$ 发送窗口+接收窗口

■ 等价性

- 收发窗口都为 1 时，三者退化为停等协议，行为等价

TCP 确认号与累积确认



- 确认号 = 期望收到的下一个字节序号 (累积确认)
- 典型场景：两段报文序号 127、207
 - 207 先到：接收方仍回 ACK=127，暂存乱序数据
 - 127 到达后：回 ACK=207 (或更高，取决于长度)
 - 首个 ACK 丢失：发送方超时重传，接收方靠序号识别为重复并丢弃

TCP 连接管理：三次握手、四次挥手、TIME_WAIT



■ 三次握手

- SYN → SYN-ACK → ACK; 同步初始序号, 确认双方收发能力

■ 四次挥手

- 连接全双工, 每个方向独立关闭 (FIN/ACK 各一次), 故比握手多一次

■ TIME_WAIT 等待 2•MSL

- 确保最后的 ACK 到达对端, 并让本连接旧报文失效, 避免影响新连接

TCP 流量控制 (rwnd) vs 拥塞控制 (cwnd)



■ 流量控制

- 保护接收方缓存不被压垮，由 rwnd（接收窗口）通告

■ 拥塞控制

- 保护网络不被压垮，由 cwnd（拥塞窗口）动态调整

■ 发送窗口 = $\min(\text{rwnd}, \text{cwnd})$

- 高速下载/接收慢 $\rightarrow$ rwnd 成瓶颈；网络拥塞 $\rightarrow$ cwnd 成瓶颈
- rwnd=0 后：发送方周期发 1 字节探测，促使接收方回送最新 rwnd

TCP 拥塞控制：慢启动 / 拥塞避免 / 快速恢复



- 慢启动：cwnd 从 1 MSS 指数增长，直到 ssthresh 或丢包
- 拥塞避免：cwnd 线性增长（每 RTT +1 MSS）
- 丢包响应（区分两种信号）
 - 3 个冗余 ACK（轻度）：ssthresh=cwnd/2, cwnd 减半 → 快速恢复（Reno）
 - 超时（重度）：ssthresh=cwnd/2, cwnd 降为 1, 重新慢启动
- Reno vs Tahoe: Tahoe 对 3 冗余 ACK 也降为 1; Reno 快速恢复吞吐更高

数据平面 vs 控制平面



■ 数据平面（每台路由器、本地、快）

- 决定到达分组如何转发：查转发表、选输出端口

■ 控制平面（全网范围、慢）

- 决定端到端路由，即如何生成转发表（路由算法）

■ 两种实现

- 分布式：每台路由器跑 OSPF/BGP；集中式 SDN：控制器统一计算下发
- 类比：转发=快递员按面单分拣；路由选择=站长规划全网线路

IP 地址与二进制转换



■ 点分十进制 ↔ 32 位二进制

- 每段 (0–255) 转成恰好 8 位, 不足补前导 0 (如 27 → 00011100)
- 四段之间用点号分隔

■ 示例

- 223.1.3.27 → 11011111 00000001 00000011 00011100

■ 网络前缀 / 主机部分

- 由子网掩码 (如 /24、/16) 划分; 可求网络地址与广播地址并验算

■ 作用

- 内网多主机共享一个公网 IP，隐藏内网拓扑，缓解 IPv4 地址枯竭

■ NAT 转换表

- (内网 IP, 端口) $\leftrightarrow$ (公网 IP, NAT 端口) ; 同时改写 IP 与端口

■ 关键场景

- 两内网主机用相同源端口访问同一 80 端口 $\rightarrow$ NAT 分配不同对外端口
- 返回报文据 NAT 端口准确交回原主机; DHCP 动态分配私有 IP

匹配 + 转发与最长前缀匹配



■ 基于目的地转发

- 仅匹配目的 IP，按最长前缀匹配（LPM）选表项进行查找

■ 通用转发 match + action (SDN)

- 可匹配多字段；动作除转发外还可丢弃、改写、复制、送控制器

■ 统一性

- match+action 统一描述路由器/交换机/防火墙/NAT，是 OpenFlow 基础

■ IPv4 分片

- 数据报大于链路 MTU 时由路由器分片
- 用标识、标志（MF）、片偏移在目的主机重组（区别于 TCP 分段）

■ IPv6 改进

- 固定 40 字节首部，取消中间路由器分片与首部校验和
- 引入流标签（Flow Label）提升转发效率；128 位地址彻底解决枯竭

OSPF 开放最短路优先协议



■ 本质：AS 内链路状态协议

■ 泛洪机制

- 每台路由器生成链路状态信息，向所有邻居扩散，全 AS 获得完整拓扑

■ 防重复与分区

- 带序号与 Age（老化）字段，重复 / 过旧的链路状态分组被丢弃
- 大型 AS 划分多区域，区域 0（骨干）连接所有其他区域

路由选择算法：链路状态 LS vs 距离矢量 DV



■ 链路状态 LS (Dijkstra)

- 每节点掌握全网拓扑（泛洪），各自算最短路；收敛快、无环、开销大

■ 距离矢量 DV (Bellman-Ford)

- 每节点只知邻居距离，迭代交换“到各目的代价”，逐步收敛

■ 无穷计数问题

- 链路失效后代价累加；毒性逆转 / 水平分割 / 最大跳数缓解
- 不能简单说 LS “更优”，二者各有适用场景

- **本质：AS 间路径矢量协议，传播可达性 + AS_PATH**

- **高频纠错**

- 向外通告时必须在 AS_PATH 前加上自身 AS 号 → 用于环路检测

- **策略选路**

- 不会把所有路径通告给所有邻居；通常只通告自己选定的最佳路径

- 选路顺序：local-pref → 最短 AS_PATH → 热土豆 → 其他规则

自治系统与层次路由：iBGP / eBGP



■ AS（自治系统）

- 统一管理策略的路由器集合；路由分为两层

■ AS 内（iBGP / 域内）

- 如 OSPF、RIP，目标是最短 / 最优路径

■ AS 间（eBGP / 域间）

- 如 BGP，目标是可达性 + 策略（经济、商业关系）
- iBGP 在 AS 内传播外部路由，与 OSPF 职责不同，不冗余

- **定位：网络层协议，承载在 IP 之上，提供差错报告与诊断**

- **常见报文类型**

- 回显请求 / 应答 (8/0, ping)、目的不可达 (3)、超时 / TTL 超期 (11)

- **ping 与 traceroute**

- ping：发回显请求收应答，测连通性与 RTT

- traceroute：TTL 从 1 逐跳递增，每跳回送 ICMP 超时 (11)，逐跳暴露路径

- 防火墙丢弃所有 ICMP：ping 超时、traceroute 对应跳显示